

LAPORAN PRAKTIKUM

Aplikasi Berbasis Platform

MODUL 4-5

Flutter



Disusun oleh:

MUHAMMAD KAMIL FAUZAN

2311102314

S1 IF-11-REG04

Dosen Pengampu:

Cahyo Prihantoro, S.Kom., M.Eng

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY

2026/2027

JAWABAN

1.Container

Source Code

```
sectionTitle('1. Container'),  
  
Container(  
  width: double.infinity,  
  height: 100,  
  alignment: Alignment.center,  
  decoration: BoxDecoration(  
    color: Colors.blue,  
    borderRadius: BorderRadius.circular(12),  
  ),  
  child: const Text(  
    'Ini adalah Container  
berwarna', style: TextStyle(  
color: Colors.white,  
fontSize: 18,  
fontWeight: FontWeight.bold,  
    ),  
  ),  
),
```

Output:



Penjelasan:

Implementasi pada bagian ini menggunakan widget **Container** untuk memvisualisasikan sebuah elemen berbentuk kotak berwarna biru. Dimensi lebar kotak diatur agar bersifat dinamis mengikuti lebar layar, sementara tingginya ditetapkan sebesar **100 piksel**. Untuk meningkatkan aspek visual, sudut pada kotak diberikan efek melengkung (*border radius*). Di dalamnya, terdapat komponen teks bertuliskan “Ini adalah Container berwarna” yang diposisikan tepat di tengah menggunakan properti perataan. Teks tersebut diformat dengan warna putih dan gaya cetak tebal (*bold*) guna menjamin keterbacaan yang optimal.

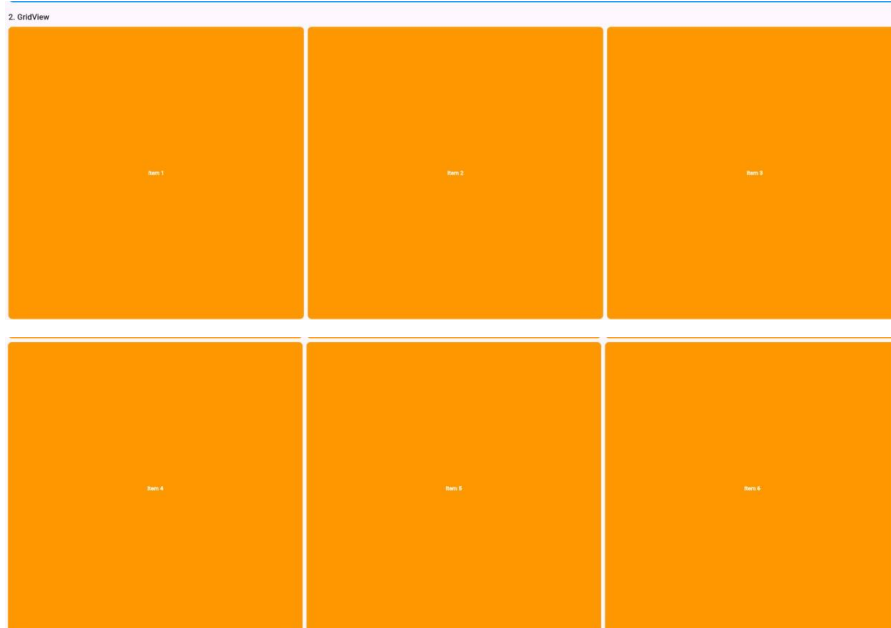
2.GridView

Source Code:

```
sectionTitle('2. GridView Minimal 6 Item'),

GridView.count(
  crossAxisCount: 3,
  shrinkWrap: true,
  physics: const
NeverScrollableScrollPhysics(),
  crossAxisSpacing: 10,  mainAxisSpacing: 10,
  children: List.generate(6, (index) {
return Container(
  alignment: Alignment.center,
  decoration: BoxDecoration(
color: Colors.orange,
    borderRadius: BorderRadius.circular(10),
  ),
  child: Text(
    'Item ${index + 1}',
style: const TextStyle(
color: Colors.white,
    fontWeight: FontWeight.bold,
  ),
),
);
  })),
),
```

Output:



Penjelasan:

Widget **GridView** berhasil digunakan untuk menampilkan beberapa item dalam bentuk susunan kotak atau grid. Widget ini cocok digunakan untuk menampilkan data yang berbentuk menu, galeri, atau daftar item dengan tampilan lebih rapi dan terstruktur.

3.ListView

Source Code:

```
sectionTitle('3. ListView'),

SizedBox(  height:
160,  child:
ListView(
children: const [
Card(
  child: ListTile(
    leading: Icon(Icons.looks_one),
title: Text('Item A'),
  ),
),
  Card(
    child: ListTile(
      leading: Icon(Icons.looks_two),
title: Text('Item B'),
    ),
  ),
  Card(
    child: ListTile(
      leading: Icon(Icons.looks_3),
title: Text('Item C'),
    ),
  ),
],
),
),
```

Output:



Penjelasan:

Implementasi bagian ini memanfaatkan widget **ListView** untuk menyajikan data secara vertikal. Daftar tersebut terdiri dari tiga elemen, yakni Item A, B, dan C, yang masing-masing dibungkus menggunakan widget **Card** untuk memberikan kesan modular dan rapi. Di dalam setiap kartu, terdapat **ListTile** yang berfungsi mengatur tata letak secara otomatis, dengan ikon di sisi kiri (*leading*) dan teks keterangan di sisi kanan. Penggunaan **ListView** sangat ideal dalam skenario ini karena kemampuannya menyusun informasi secara berurutan dari atas ke bawah.

4.ListView.builder

Source Code

```
sectionTitle('4. ListView.builder'),

SizedBox(
  height: 250,
  child: ListView.builder(
    itemCount: dataBuah.length,
    itemBuilder: (context, index) {
      return Card(
        child: ListTile(
          leading: const Icon(Icons.apple),
          title: Text(dataBuah[index]),
          subtitle: Text('Data buah ke-${index + 1}'),
        ),
      );
    },
  ),
),
```

Bagian data array-nya:

```
final List<String> dataBuah = const [  
    'Apel',  
    'Jeruk',  
    'Mangga',  
    'Pisang',  
    'Semangka',  
];
```

Output:



Penjelasan:

Pada bagian ini, widget **ListView.builder** diimplementasikan untuk melakukan iterasi terhadap data dari sebuah *array* yang berisi daftar nama buah, seperti Apel, Jeruk, Mangga, Pisang, dan Semangka. Setiap elemen data dibungkus menggunakan kombinasi widget **Card** dan **ListTile** untuk menciptakan tampilan yang sistematis. Struktur visualnya mencakup ikon pada sisi kiri serta nama buah dan informasi indeks urutan pada sisi kanan. Penggunaan **ListView.builder** memberikan efisiensi memori yang lebih baik karena widget hanya akan dibuat secara dinamis sesuai dengan jumlah data yang tersedia di dalam *array*.

5.ListView.separated dengan Garis Pembatas Source

Code:

```
sectionTitle('5. ListView.separated dengan Garis Pembatas'),
SizedBox(
height: 250,
  child: ListView.separated(
itemCount: dataHewan.length,
itemBuilder: (context, index) {
return ListTile(
  leading: const Icon(Icons.pets),
title: Text(dataHewan[index]),
  );
},
  separatorBuilder: (context, index) {
return const Divider(          color:
Colors.grey,          thickness: 1,
  );
},
),
),
```

Bagian data array-nya:

```
final List<String> dataHewan = const [
  'Kucing',
  'Anjing',
  'Burung',
  'Ikan',
  'Kelinci',
];
```

Output:

5. ListView.separated dengan Garis Pembatas

 Kucing

 Anjing

 Burung

 Ikan

Penjelasan:

Bagian ini mengimplementasikan widget **ListView.separated** untuk menyajikan daftar data hewan, yang meliputi Kucing, Anjing, Burung, Ikan, dan Kelinci, secara vertikal. Setiap elemen data ditampilkan melalui **ListTile** dengan komposisi ikon di sisi kiri dan nama hewan di sisi kanan. Karakteristik utama yang membedakan **ListView.separated** dengan versi standar adalah adanya komponen pembatas (*divider*) di antara tiap item. Kehadiran garis ini memberikan batasan visual yang jelas sehingga antarmuka daftar terlihat lebih terorganisir dan mudah dipahami oleh pengguna.

6.Stack Tampilan Bertumpuk

Source Code:

```
sectionTitle('6. Stack Tampilan Bertumpuk'),
  Center(
    child: Stack(
      alignment: Alignment.center,
      children: [
        Container(
          width: 220,
          height: 150,
          decoration: BoxDecoration(
            color: Colors.green,
            borderRadius: BorderRadius.circular(16),
          ),
        ),
        Container(
          width: 160,
          height: 100,
          decoration: BoxDecoration(
            color: Colors.yellow,
            borderRadius: BorderRadius.circular(16),
          ),
        ),
        const Text(
          'Ini Stack',
          style: TextStyle(
            fontSize: 22,
            fontWeight: FontWeight.bold,
            color: Colors.black,
          ),
        ),
      ],
    ),
  ),
),
```


Output:

6. Stack Tampilan Bertumpuk



Penjelasan:

Implementasi pada bagian ini menggunakan widget **Stack** untuk menciptakan komposisi visual yang saling bertumpang tindih. Struktur tumpukan tersebut terdiri dari dua buah **Container**, yakni kotak berwarna hijau pada lapisan belakang dan kotak kuning pada lapisan depan. Di atas lapisan teratas, disematkan teks bertuliskan “Ini Stack”. Seluruh elemen tersebut diposisikan secara presisi di bagian tengah melalui konfigurasi *alignment*. Penggunaan **Stack** sangat efektif ketika pengembang ingin menyusun beberapa widget dalam satu area koordinat yang sama secara berlapis.